

Практическое ДЗ: построение REST API на Go

Цель

Написать небольшой HTTP API для управления пользователями. Данные должны храниться в памяти приложения в Go-мапе. Базу данных подключать не нужно.

В ходе работы нужно потренироваться:

- создавать HTTP-сервер на Go;
- регистрировать обработчики маршрутов;
- получать параметры из URL;
- читать JSON из тела запроса;
- преобразовывать JSON в Go-структуру;
- возвращать JSON и корректные HTTP-статусы;
- хранить и удалять данные в `map`.

Технические требования

- Язык: Go.
- Можно использовать стандартную библиотеку `net/http`.
- Данные хранятся только в памяти приложения.
- Формат запросов и ответов: JSON.
- Сервер должен запускаться, например, на порту `8080`.
- После перезапуска приложения данные могут пропадать.
- Код каждого HTTP-обработчика желательно вынести в отдельную функцию.

Модель пользователя

Пользователь должен содержать следующие поля:

```
type User struct {
    ID        int    `json:"id"`
    Name      string `json:"name"`
    Email     string `json:"email"`
    Password  string `json:"password,omitempty"`
}
```

Для хранения пользователей можно использовать мапу:

```
var users = make(map[int]User)
```

ID нового пользователя приложение должно генерировать самостоятельно. Например, можно хранить отдельный счётчик и увеличивать его после каждого создания пользователя.

В реальном API пароль нельзя хранить в открытом виде. Его необходимо хешировать. В рамках этого учебного задания допускается временно хранить пароль как строку, но пароль нельзя возвращать клиенту в ответах `GET` и `POST`.

Отдельное упражнение перед реализацией `POST /user`

Сначала напишите небольшой обработчик, который получает JSON из тела HTTP-запроса и переносит данные в Go-структуру.

Пусть клиент отправляет такой JSON:

```
{
  "name": "Алексей",
  "email": "alex@example.com",
  "password": "secret123"
}
```

Структура для входных данных:

```
type CreateUserRequest struct {
    Name      string `json:"name"`
    Email     string `json:"email"`
    Password  string `json:"password"`
}
```

Пример получения данных из `body`:

```
func decodeUserExample(w http.ResponseWriter, r *http.Request) {
    var input CreateUserRequest

    err := json.NewDecoder(r.Body).Decode(&input)
    if err != nil {
        http.Error(w, "invalid JSON", http.StatusBadRequest)
        return
    }

    fmt.Printf("name: %s\n", input.Name)
    fmt.Printf("email: %s\n", input.Email)
    fmt.Printf("password: %s\n", input.Password)
```

```
w.WriteHeader(http.StatusNoContent)
}
```

Для работы примера понадобятся импорты:

```
import (
    "encoding/json"
    "fmt"
    "net/http"
)
```

Обратите внимание на `&input`: метод `Decode` должен получить указатель на структуру, чтобы записать в неё значения из JSON.

После выполнения упражнения перенесите этот подход в обработчик `POST /user`.

API, которое нужно реализовать

1. `GET /users` — получить всех пользователей

Обработчик должен:

1. Получить всех пользователей из мапы.
2. Сформировать массив пользователей.
3. Не включать пароли в ответ.
4. Вернуть массив в формате JSON со статусом `200 OK`.

Пример ответа:

```
[
  {
    "id": 1,
    "name": "Алексей",
    "email": "alex@example.com"
  },
  {
    "id": 2,
    "name": "Мария",
    "email": "maria@example.com"
  }
]
```

Если пользователей нет, API должен вернуть пустой массив:

```
[]
```

2. `GET /user/{id}` — получить пользователя по ID

Обработчик должен:

1. Получить `id` из URL.
2. Преобразовать его из строки в число.
3. Найти пользователя в мапе.
4. Вернуть пользователя без пароля и статус `200 OK`.

Если `id` имеет неверный формат, вернуть:

- статус `400 Bad Request` ;
- JSON с описанием ошибки.

Если пользователь не найден, вернуть:

- статус `404 Not Found` ;
- JSON с описанием ошибки.

Пример успешного ответа:

```
{
  "id": 1,
  "name": "Алексей",
  "email": "alex@example.com"
}
```

3. `POST /user` — создать пользователя

Обработчик должен:

1. Прочитать JSON из тела запроса.
2. Перенести данные в `CreateUserRequest`.
3. Проверить, что `name`, `email` и `password` заполнены.
4. Сгенерировать новый уникальный ID.
5. Создать значение типа `User`.
6. Сохранить пользователя в мапу по его ID.
7. Вернуть созданного пользователя без пароля и статус `201 Created`.

Пример запроса:

```
{
  "name": "Алексей",
  "email": "alex@example.com",
  "password": "secret123"
}
```

Пример ответа:

```
{
  "id": 1,
```

```
"name": "Алексей",  
"email": "alex@example.com"  
}
```

Если JSON некорректен или обязательное поле отсутствует, вернуть статус `400 Bad Request` и JSON с ошибкой.

4. **DELETE /users** — удалить всех пользователей

Обработчик должен:

1. Удалить все записи из мапы.
2. Вернуть статус `204 No Content`.

При статусе `204` тело ответа отправлять не нужно.

5. **DELETE /user/{id}** — удалить пользователя по ID

Обработчик должен:

1. Получить и проверить `id` из URL.
2. Проверить наличие пользователя в мапе.
3. Удалить найденного пользователя.
4. Вернуть статус `204 No Content`.

Если пользователь не найден, вернуть статус `404 Not Found` и JSON с ошибкой.

Формат ошибок

Все ошибки API оформляйте одинаково:

```
{  
  "error": "user not found"  
}
```

Для этого можно создать отдельную структуру:

```
type ErrorResponse struct {  
    Error string `json:"error"`  
}
```

Заголовки ответа

Во всех ответах с JSON устанавливайте заголовок до вызова `WriteHeader`:

```
w.Header().Set("Content-Type", "application/json")
```

Для записи структуры в ответ используйте:

```
json.NewEncoder(w).Encode(value)
```

Примеры проверки через `curl`

Создание пользователя:

```
curl -i -X POST http://localhost:8080/user \
-H 'Content-Type: application/json' \
-d '{"name":"Алексей","email":"alex@example.com","password":"secret123"}'
```

Получение всех пользователей:

```
curl -i http://localhost:8080/users
```

Получение пользователя с ID `1` :

```
curl -i http://localhost:8080/user/1
```

Удаление пользователя с ID `1` :

```
curl -i -X DELETE http://localhost:8080/user/1
```

Удаление всех пользователей:

```
curl -i -X DELETE http://localhost:8080/users
```

Ожидаемая структура проекта

Минимальный вариант:

```
users-api/
├─ go.mod
└─ main.go
```

При желании обработчики и модели можно разделить:

```
users-api/
├─ go.mod
├─ main.go
```

Критерии готовности

Работа считается выполненной, если:

- приложение запускается без ошибок;
- реализованы все пять маршрутов;
- `POST /user` читает JSON из тела запроса и добавляет пользователя в мапу;
- созданному пользователю присваивается уникальный ID;
- `GET /users` возвращает всех пользователей;
- `GET /user/{id}` возвращает нужного пользователя;
- оба маршрута `DELETE` корректно удаляют данные;
- API возвращает подходящие HTTP-статусы;
- ошибки возвращаются в формате JSON;
- пароль не попадает в ответы API;
- код отформатирован командой `go fmt`.

Дополнительные задания со звёздочкой

1. Проверить уникальность email при создании пользователя. При повторном email возвращать `409 Conflict`.
2. Добавить проверку формата email.
3. Ограничить максимальный размер тела запроса.
4. Запретить неизвестные поля в JSON через `Decoder.DisallowUnknownFields()`.
5. Защитить мапу от одновременной записи из нескольких запросов с помощью `sync.RWMutex`.
6. Написать тесты обработчиков с использованием `net/http/httptest`.
7. Разделить HTTP-модель запроса, внутреннюю модель пользователя и модель ответа на разные структуры.

Что нужно сдать

- исходный код проекта;
- короткий `README.md` с командой запуска;
- примеры запросов или скриншоты проверки всех маршрутов;
- ответы на два вопроса:
 - i. Почему в `Decode(&input)` перед структурой ставится `&`?
 - ii. Чем отличаются статусы `200`, `201`, `204`, `400` и `404`?